

GOSIM Paris 2026

May 5-6, 2026 Station F, Paris



Khal and Inferi: Towards Single-Language Cross-Platform GPU Inference with Rust and rust-gpu

Sébastien Crozet — sebcrozet@dimforge.com



Writing tensor operators on
the GPU is hard

($\circ$ $\square$ $\circ$) $\circ$ $\perp$ $\perp$ $\perp$

Dominated by *per-backend
implementations*

Can we
“Write once run anywhere”

LINEAR
ALGEBRA

SCIENTIFIC
COMPUTING

GPU SCIENTIFIC
COMPUTING

CROSS-PLATFORM
GPU
SCIENTIFIC COMPUTING



imgflip.com



wgmath/wgml

slang-hal/slai

khal/inferi

2024



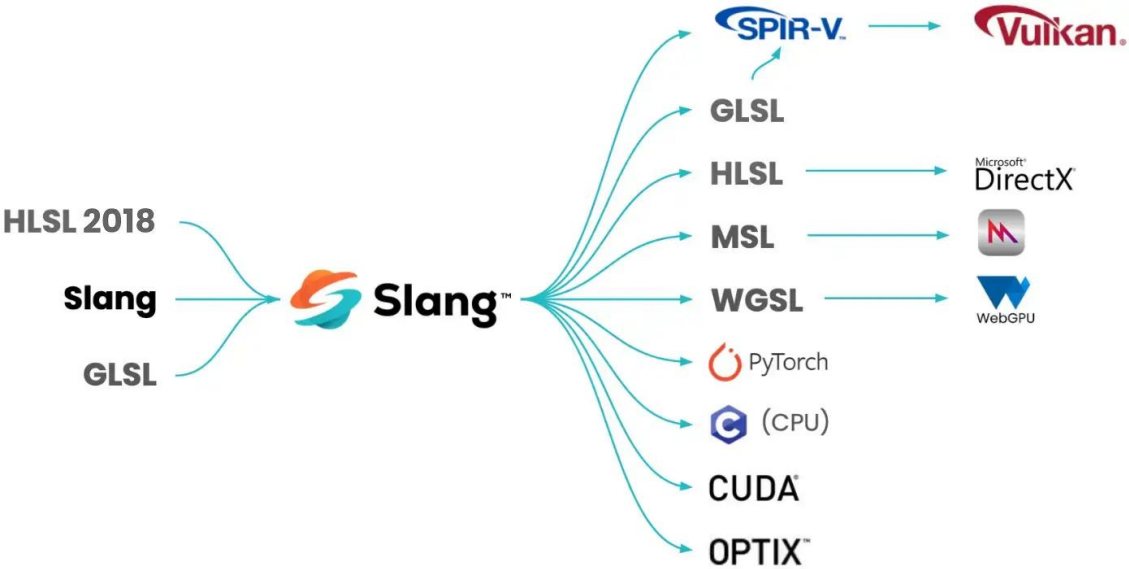
Supported end-points

Front-end	Status	Feature	Notes
SPIR-V (binary)	✓	spv-in	
WGSL	✓	wgsl-in	Fully validated
GLSL	OK	glsl-in	GLSL 440+ and Vulkan semantics only

Back-end	Status	Feature	Notes
SPIR-V	✓	spv-out	
WGSL	OK	wgsl-out	
Metal	✓	msl-out	
HLSL	✓	hlsl-out	Shader Model 5.0+ (DirectX 11+)
GLSL	OK	glsl-out	GLSL 330+ and GLSL ES 300+
AIR			
DXIL/DXIR			
DXBC			
DOT (GraphViz)	OK	dot-out	Not a shading language

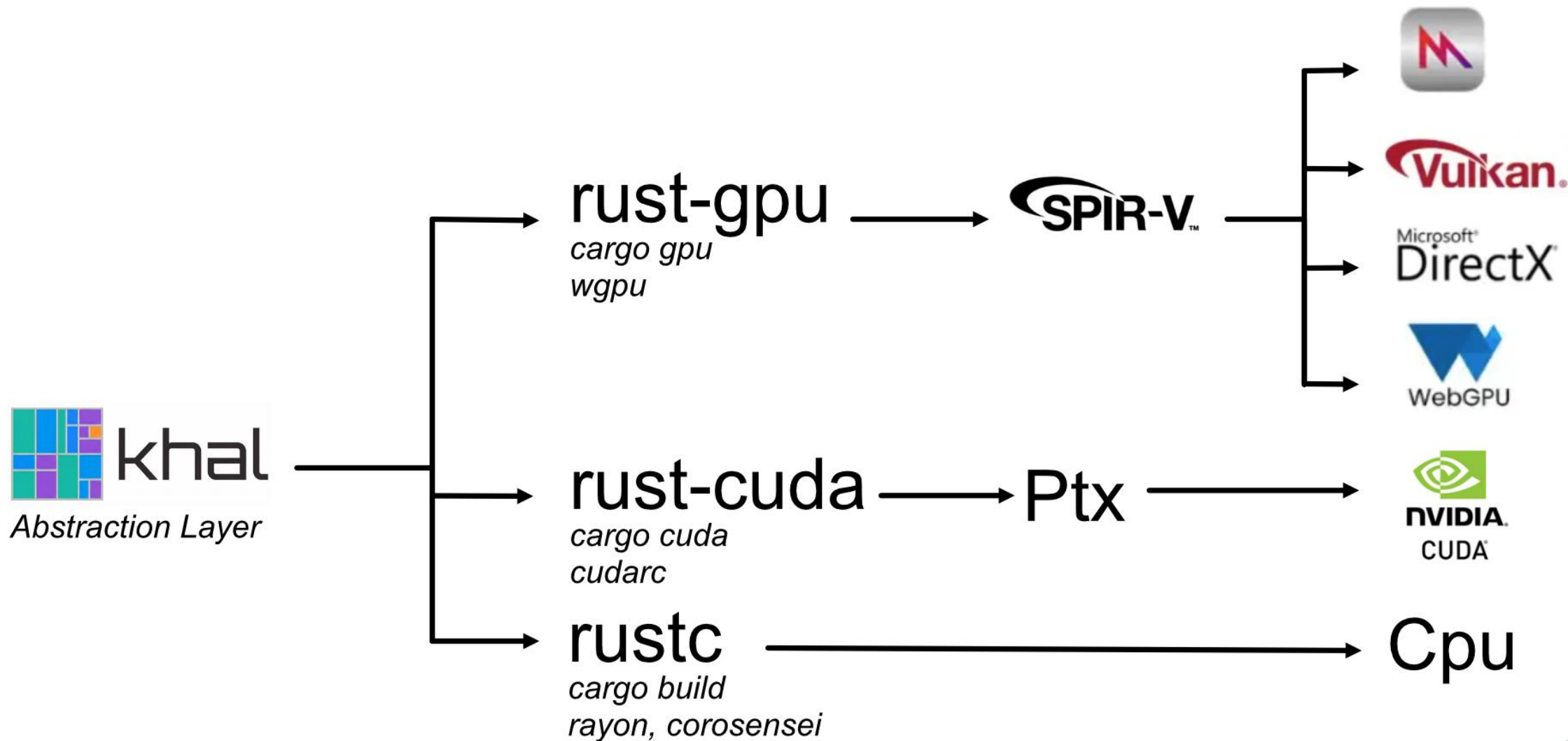
✓ = Primary support — OK = Secondary support — — = Unsupported, but support in progress

2025



2026–Today

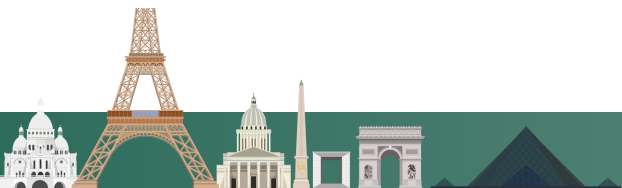




```
#![cfg_attr(target_arch = "spirv", no_std)]

use khal_std::glamx::UVec3;
use khal_std::macros::{spirv_bindgen, spirv};

#[spirv_bindgen]
#[spirv(compute(threads(64)))]
pub fn gpu_add_assign(
    #[spirv(global_invocation_id)] invocation_id: UVec3,
    #[spirv(storage_buffer, descriptor_set = 0, binding = 0)] a: &mut [f32],
    #[spirv(storage_buffer, descriptor_set = 0, binding = 1)] b: &[f32],
) {
    let thread_id = invocation_id.x as usize;
    if thread_id < a.len() {
        a[thread_id] += b[thread_id];
    }
}
```



```
use khal::backend::{Backend, Buffer, Encoder, GpuBackend, GpuBackendError, WebGpu};
use khal::re_exports::include_dir::{Dir, include_dir};
use khal::{BufferUsages, Shader};
use khal_example_shaders::AddAssign;

static SPIRV_DIR: Dir<'static> = include_dir!("$CARGO_MANIFEST_DIR/shaders-spirv");

#[derive(Shader)]
pub struct GpuKernels {
    add_assign: AddAssign,
}

#[async_std::main]
async fn main() {
    let webgpu = WebGpu::default().await.unwrap();
    let backend = GpuBackend::WebGpu(webgpu);

    // Run the operation and display the result.
    let a = (0..10000).map(|i| i as f32).collect::<Vec<_>>();
    let b = (0..10000).map(|i| i as f32 * 10.0).collect::<Vec<_>>();
    let result = compute_sum(&backend, &a, &b).await.unwrap();
    println!("Computed sum: {result:?}");
}

async fn compute_sum(
    backend: &GpuBackend,
    a: &[f32],
    b: &[f32],
) -> Result<Vec<f32>, GpuBackendError> {
    // Generate the GPU buffers.
    let mut a = backend.init_buffer(a, BufferUsages::STORAGE | BufferUsages::COPY_SRC)?;
    let b = backend.init_buffer(b, BufferUsages::STORAGE)?;

    // Dispatch the operation on the gpu.
    let kernels = GpuKernels::from_backend(backend)?;
    let mut encoder = backend.begin_encoding();
    let mut pass = encoder.begin_pass("add_assign", None);
    kernels.add_assign.call(&mut pass, a.len(), &mut a, &b)?;
    drop(pass);
    backend.submit(encoder)?;

    // Read the result (slower but convenient version).
    backend.slow_read_vec(&a).await
}
```



1

Type-safe by construction

- `#[spirv_bindgen]` reflects shaders into typed Rust APIs at compile time.
- Buffer / kernel signature mismatches → **compile errors**.
- Binding groups & indices bound automatically.

2

Debug like normal Rust

- Run the same shader on the CPU (rayon + coroutines).
- Breakpoints, prints, sanitizers, like regular Rust CPU code.
- If it works on CPU, ship the same code to GPU.

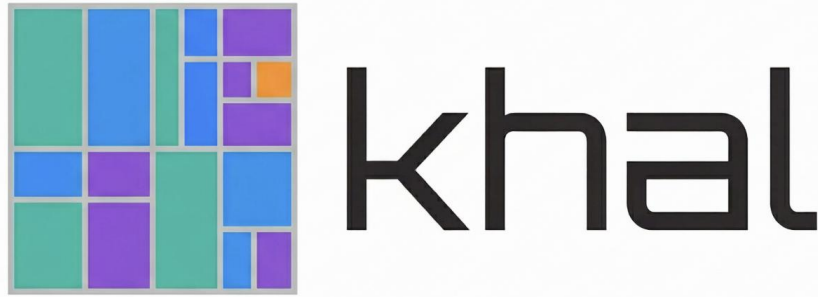
3

One language, every backend

- Shaders are `no_std` Rust crates — share kernels via Cargo.
- High-level Rust language constructs.

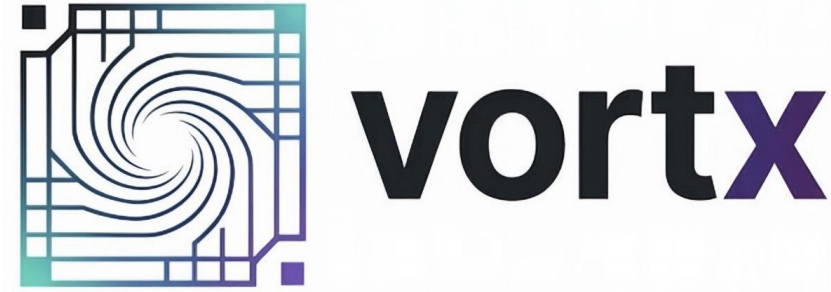


Abstraction layer



<https://github.com/dimforge/khal>

Tensor library



<https://github.com/dimforge/vortex>

Inference library



<https://github.com/dimforge/inferi>

Physics - Datasets generation



Planned for 2026



Thanks!

